Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS

Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No: 21601206
Name: Chinmay Yannawar

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # 1. Create linked list
    def create(self, values):
        self.head = None
        tail = None
        for v in values:
            node = Node(v)
            if self.head is None:
                self.head = tail = node
            else:
                tail.next = node
                tail = node
```

```
# 2. Traverse and print
    def display(self):
        if self.head is None:
            print("List is empty")
            return
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

    # 3. Insert at position (1-based)
    def insert_at(self, pos, data):
        if pos < 1:
            print("Invalid position")
            return
        node = Node(data)
        if pos == 1:
            node.next = self.head
            self.head = node
            return
        temp = self.head
        for _ in range(pos - 2):
            if temp is None:
                break
            temp = temp.next
        if temp is None:
            print("Position out of range")
            return
        node.next = temp.next
        temp.next = node

    # 4. Find middle node
    def find_middle(self):
        if self.head is None:
```

```
                print("List is empty")
            return
        slow = fast = self.head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        print("Middle node value:", slow.data)


# 5. Delete node at position (1-based)
    def delete_at(self, pos):
        if self.head is None:
            print("List is empty")
            return
        if pos < 1:
            print("Invalid position")
            return
        if pos == 1:
            self.head = self.head.next
            return
        temp = self.head
        for _ in range(pos - 2):
            if temp is None:
                break
            temp = temp.next
        if temp is None or temp.next is None:
            print("Position out of range")
            return
        temp.next = temp.next.next
```

```
# 6. Reverse list
    def reverse(self):
        prev = None
        curr = self.head
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
        self.head = prev

    # 7. Sum of every two consecutive nodes
    def sum_consecutive(self):
        if self.head is None or self.head.next is None:
            print("Need at least 2 nodes")
            return
        temp = self.head
        while temp.next:
            print(f"{temp.data} + {temp.next.data} = {temp.data + temp.next.data}")
            temp = temp.next

def solution():
    ll = SinglyLinkedList()

    ll.create([10, 20, 30, 40])
    print("List:")
    ll.display()

    ll.insert_at(3, 25)
    print("After inserting 25 at position 3:")
    ll.display()

    ll.find_middle()

    ll.delete_at(2)
    print("After deleting position 2:")
```

```
ll.display()

    print("Consecutive sums:")
    ll.sum_consecutive()

    ll.reverse()
    print("After reversing:")
    ll.display()

solution()
```

```
PS C:\Users\Admin\Desktop\DSA> & C:\Users\
ssignment/demo.py
List:
10 -> 20 -> 30 -> 40 -> None
After inserting 25 at position 3:
10 -> 20 -> 25 -> 30 -> 40 -> None
Middle node value: 25
After deleting position 2:
10 -> 25 -> 30 -> 40 -> None
Consecutive sums:
10 + 25 = 35
25 + 30 = 55
30 + 40 = 70
After reversing:
40 -> 30 -> 25 -> 10 -> None
PS C:\Users\Admin\Desktop\DSA>
```